# PROVISIONAL PATENT APPLICATION

## SYSTEM AND METHOD FOR GUARANTEEING CORRECTNESS AND SAFETY OF USER-SUBMITTED CODE EXECUTING INSIDE A DATABASE THROUGH SANDBOXED WEBASSEMBLY MODULES WITH CONTENT-ADDRESSED VERIFICATION

Authors: Tom Tang

Organization: Edge Vector Foundation

Date: March 30, 2026

---

## Applicant

Edge Vector Foundation
2328 Delmar Dr E, Seattle, WA 98102

## Inventors

- Tom Tang, Seattle, WA

## Field of the Invention

This invention relates to verified execution of user-submitted code within a database system, and more particularly to a system and method for guaranteeing correctness, safety, and termination of WebAssembly transforms executing inside a database through content-addressed identity, sandboxed execution, gas metering, deterministic semantics, and registration-time type verification.

## Cross-Reference to Related Applications

None (provisional filing).

## Background of the Invention

Databases have long supported computed fields and derived views. SQL expressions, stored procedures, and user-defined functions allow applications to define computations that run inside the

database engine. However, these mechanisms suffer from fundamental limitations when user-submitted code is involved.

In traditional database systems, stored procedures and user-defined functions execute in the database's own runtime environment. They have access to the filesystem, network, other tables, and system resources. This makes user-submitted code a severe security risk: a malicious or buggy function can exfiltrate data, corrupt other records, exhaust system resources through infinite loops, or open network connections to external servers. As a result, most production databases either prohibit user-submitted code entirely or restrict it to a narrow set of built-in operations.

Systems that do allow user-submitted code typically execute it in a separate process with inter-process communication (IPC) overhead. This introduces latency, memory duplication, and serialization costs that make per-field transforms impractical at scale.

Beyond security, existing systems provide no mechanism to verify that the code a user approved is the code that actually executes. A stored procedure can be silently modified after approval. There is no tamper detection, no immutability guarantee, and no way for third parties to independently verify what computation was performed.

Additionally, existing systems perform no registration-time verification of type compatibility. A stored procedure that expects an integer but receives a string will fail at runtime, potentially after processing millions of records. There is no mechanism to verify before any data flows that a transform's input and output types are compatible with the fields it connects.

There is a need for a system that enables user-submitted code to execute inside a database with provable safety guarantees: verified identity, sandboxed execution, guaranteed termination, deterministic results, and type-checked interfaces — all verified before any data flows through the transform.

## Summary of the Invention

The present invention provides a system and method for verified execution of user-submitted code inside a database, comprising:

1. **Content-addressed transform identity** — each transform is identified by the SHA-256 hash of its WebAssembly bytecode, providing immutability, tamper detection, deduplication, and public auditability.
2. **Sandboxed execution with zero system access** — WASM modules execute in a runtime with no filesystem, network, clock, or system access, and no access to database records or host memory outside the module's own linear memory.
3. **Gas metering for termination guarantee** — each module is assigned a fuel budget; every WASM instruction consumes fuel; when fuel is exhausted, execution aborts, guaranteeing termination without infinite loops or resource exhaustion.
4. **Deterministic execution** — WASM with NaN canonicalization produces identical output for identical input on every platform, enabling reproducible and independently verifiable transform results.

5. **Registration-time type verification** — before any data flows through a transform, the system verifies input/output type compatibility, import resolution, import graph acyclicity, gas limit bounds, and optional invertibility via round-trip testing.
6. **Host-guest data passing protocol** — a defined protocol for passing typed data between the database host and the WASM guest through serialization, memory allocation within the WASM linear memory, and deserialization, ensuring the module cannot access host memory.
7. **Composable transform imports with DAG verification** — transforms can import other transforms via WASM import tables, with the import graph verified as a directed acyclic graph at registration time, and the composition hash including the hashes of all imports for full chain verifiability.

# Detailed Description of the Invention

## System Architecture

The system comprises three principal components: (1) a database engine that stores and queries data, (2) a WASM transform execution engine embedded within the database, and (3) a content-addressed transform registry that stores verified WASM modules.

### Content-Addressed Transform Identity

Each transform is identified by `SHA-256(wasm_bytes)`. The hash IS the identity — there is no separate name or version number that could become inconsistent. This content-addressing provides:

- **Immutability:** A registered transform cannot be modified. Any change to the bytecode produces a different hash, creating a new transform identity. The original identity always refers to the original code.
- **Tamper detection:** Before every execution, the database node recomputes `SHA-256(wasm_bytes)` and compares it to the expected hash. If the bytes have been altered in storage or transit, the hash will not match and execution is rejected.
- **Deduplication:** Identical WASM bytecode from different authors, submitted at different times, produces the same hash. The registry stores only one copy.
- **Public auditability:** A developer publishes source code alongside the registered hash. Any third party can independently compile the source to WASM, compute SHA-256 of the output, and verify it matches the registered hash. This provides cryptographic proof that the published source produces the registered transform.

### Sandboxed Execution Model

The WASM transform engine executes modules in a sandboxed runtime with the following constraints:

- **No filesystem access:** The module cannot read or write files on the host system. No WASI filesystem capabilities are provided.
- **No network access:** The module cannot open sockets, make HTTP requests, or communicate with any external system. No WASI network capabilities are provided.
- **No clock or time access:** The module cannot read the system clock, measure elapsed time, or access any time-related functions. This prevents timing-based side channels and ensures determinism.
- **No access to other database records:** The module receives only the serialized input bytes for the specific field being transformed. It cannot query other fields, other records, or other schemas.
- **No access to host memory:** The module operates within its own WASM linear memory. It cannot read or write any memory outside this sandbox. The host copies data into the WASM linear memory and reads results from it, but the module cannot access host memory.
- **Per-module memory limits:** Each module's linear memory is bounded to a configured maximum, preventing memory exhaustion attacks.

The module can only: (1) read the serialized input bytes that the host copied into its linear memory, (2) perform computation using its own code and linear memory, and (3) write serialized output bytes into its linear memory for the host to read.

#### Gas Metering (Fuel-Based Termination Guarantee)

Each WASM module is assigned a maximum fuel budget at registration time. The fuel limit is part of the transform's metadata and is verified to be within system-configured bounds during registration.

During execution, the WASM runtime consumes fuel for every instruction executed. When the fuel budget is exhausted, execution immediately aborts with a fuel-exhaustion error. This provides a hard termination guarantee:

- **No infinite loops:** Any loop that does not terminate within the fuel budget is aborted.
- **No resource exhaustion:** CPU consumption is bounded by the fuel budget regardless of the module's behavior.
- **Predictable execution cost:** The fuel budget provides an upper bound on execution time, enabling the database to schedule and prioritize transform execution.
- **Declared at registration:** The fuel limit is declared when the transform is registered, not at execution time. Users approving a transform can see its maximum execution cost before any data flows through it.

### Deterministic Execution

The WASM runtime is configured for deterministic execution:

- **NaN canonicalization:** All NaN floating-point values are canonicalized to a single representation, eliminating platform-dependent NaN bit patterns.
- **No system-dependent behavior:** Without access to the clock, filesystem, network, or any system state, the module's output depends solely on its input bytes and its own code.
- **Platform independence:** WASM's specification guarantees identical semantics across all conforming runtimes. Combined with NaN canonicalization, this means the same module with the same input produces the same output on any platform.
- **Verifiable results:** Because execution is deterministic, any party with the WASM module and the input can independently reproduce and verify the output.

### Registration-Time Verification Pipeline

Before any data flows through a transform, the system performs a multi-step verification pipeline at registration time:

1. **Hash computation:** Compute `SHA-256(wasm_bytes)` to derive the transform's identity.
2. **Deduplication check:** If a transform with the same hash is already registered, return the existing registration (identical code needs no re-verification).
3. **Type compatibility verification:** Verify that the transform's declared input type matches the source field's type and the declared output type matches the target field's type. Type mismatches are rejected before any data is processed.
4. **Import resolution:** Verify that all transforms referenced in the WASM module's import table exist in the registry.
5. **Acyclicity verification:** Verify that the import graph (this transform plus all transitive imports) forms a directed acyclic graph. Circular dependencies are rejected.
6. **Gas limit validation:** Verify that the declared maximum fuel is within system-configured bounds.
7. **Reversibility verification (optional):** If the transform declares itself reversible by exporting an inverse function, the system runs round-trip testing: generate type-appropriate synthetic inputs, run the forward transform, run the inverse transform on the forward output, and verify equality with the original input (floating-point tolerance $1 \times 10^{-10}$). If the round-trip fails, the transform is rejected as non-reversible.

Only transforms that pass all verification steps are registered. The hash becomes the immutable identity, and the verified metadata (types, gas limit, reversibility status, import hashes) is stored alongside the WASM bytes.

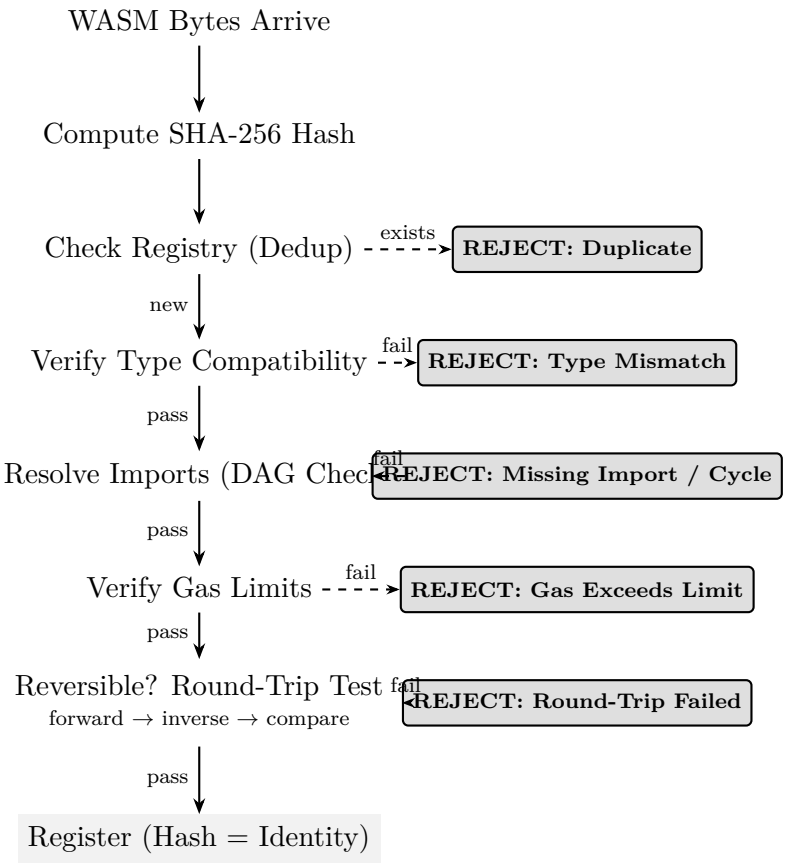


Figure 1: **Registration-Time Verification Pipeline.** WASM bytes pass through a multi-step verification pipeline before registration. Each step can reject the transform. Only transforms that pass all checks are registered with their SHA-256 hash as identity.

### Host-Guest Data Passing Protocol

Data is passed between the database (host) and the WASM module (guest) through a defined protocol that ensures the module never accesses host memory:

1. **Serialization:** The host serializes the input `FieldValue` to bytes using a deterministic binary format (bincode or msgpack).
2. **Allocation:** The host calls the module's exported `alloc(size) -> ptr` function, which allocates space within the WASM linear memory and returns a pointer.
3. **Copy-in:** The host copies the serialized bytes into the WASM linear memory at the returned pointer.
4. **Invocation:** The host calls the module's exported `transform(ptr, len) -> output_ptr` function, passing the pointer and length of the input bytes.
5. **Guest computation:** The module deserializes the input, performs its computation, serializes the output, and returns a pointer to the output bytes in its linear memory.

6. **Copy-out:** The host reads the output bytes from the WASM linear memory.
7. **Deserialization:** The host deserializes the output bytes back to a `FieldValue`.

Each WASM module exports the following functions:

- `alloc(size: u32) -> u32` — allocates `size` bytes in WASM linear memory, returns pointer
- `transform(input_ptr: u32, input_len: u32) -> u32` — executes the forward transform
- `inverse(input_ptr: u32, input_len: u32) -> u32` (optional) — executes the inverse transform for reversible modules

The output pointer returned by `transform` or `inverse` encodes both the pointer and length of the output using a convention (e.g., the first 4 bytes at the pointer contain the length, followed by the output bytes).

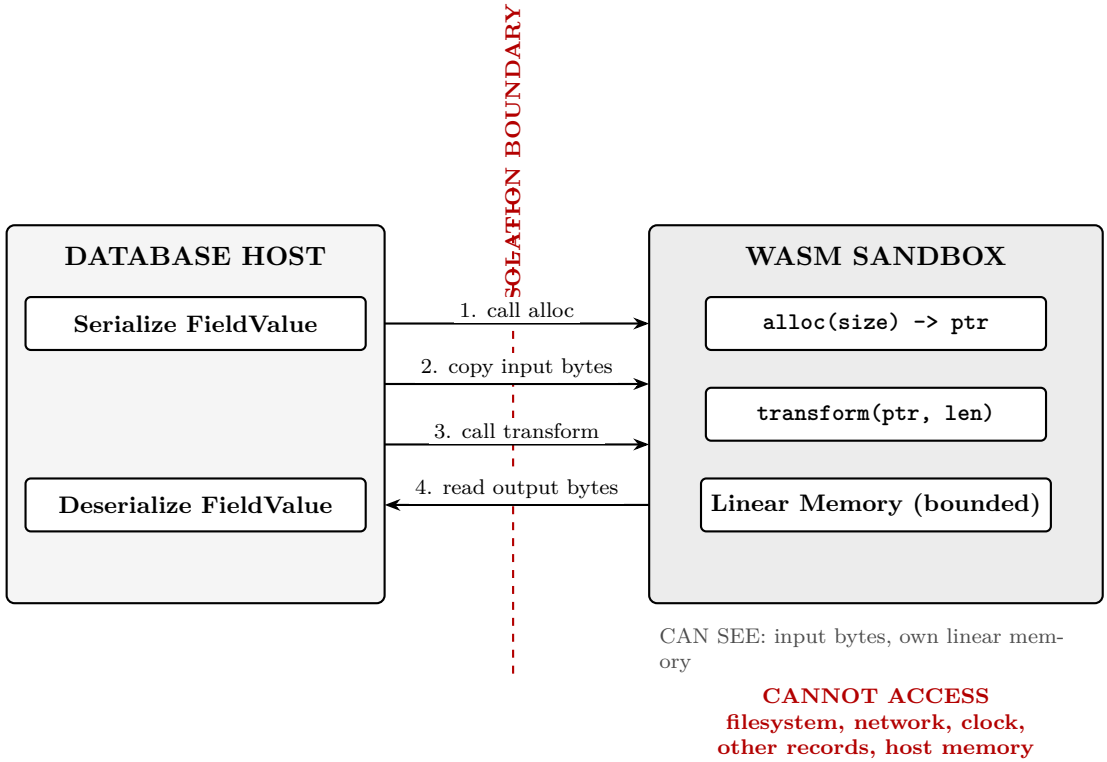


Figure 2: **Sandboxed Execution Model.** The database host serializes input, copies it into the WASM sandbox via `alloc()`, calls `transform()`, and reads the output. The sandbox can only see its input bytes and its own linear memory. It cannot access the filesystem, network, clock, other database records, or host memory.

### Composable Transform Imports

Transforms can import other transforms via WASM import tables. When a transform declares imports, the database resolves them from the registry at registration time. The import mechanism enables:

- **Code reuse:** Common operations (arithmetic, string manipulation, aggregation) are registered once as standard library transforms and imported by many composed transforms.
- **Composition:** A transform can chain multiple imported transforms, passing the output of one as the input to the next.
- **Content-addressed composition:** The composed transform's hash includes the hashes of all its imports via the WASM import table declarations. This means the full computation chain — not just the top-level module — is cryptographically identified.
- **DAG verification:** At registration time, the system traverses the full import graph and verifies it is a directed acyclic graph. Circular dependencies (A imports B, B imports A) are rejected.

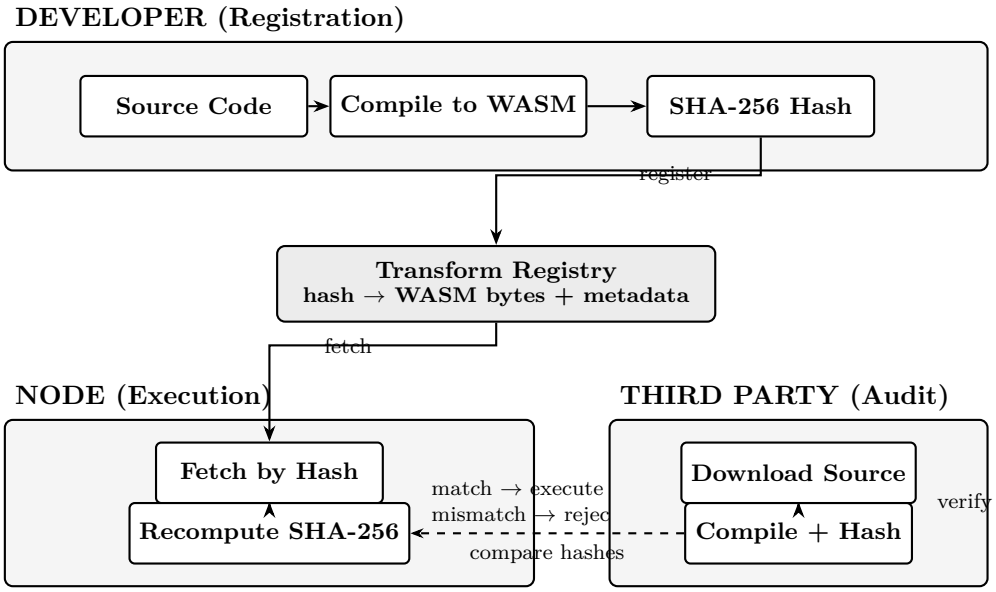


Figure 3: **Content-Addressed Verification.** A developer compiles source to WASM and registers it by hash. A node fetches the WASM by hash and recomputes the hash before execution. A third party can independently compile the published source and verify the hash matches, providing cryptographic proof of what code is executing.

### Reversibility Verification

Transforms may optionally declare reversibility by exporting an `inverse` function in addition to the `transform` function. Reversibility is verified at registration time, not asserted by the developer:

1. The system generates type-appropriate synthetic test inputs (e.g., random floats for Float fields, random strings for String fields, random arrays for Array fields).
2. For each test input, the system runs the forward transform to produce an output.
3. The system runs the inverse transform on the forward output to produce a reconstructed input.
4. The system compares the reconstructed input to the original. For floating-point values, equality is checked with tolerance $1 \times 10^{-10}$. For all other types, exact equality is required.
5. If any test input fails the round-trip, the transform is rejected as non-reversible.

Only transforms that pass round-trip testing are marked as reversible in the registry. This verified reversibility enables write-back through the transform: when a user writes a value to a transform field, the database runs the inverse transform to compute the corresponding source value and writes it to the source field.

### Deterministic Execution Guarantees

The combination of WASM semantics, NaN canonicalization, and the absence of any system access produces fully deterministic execution. For any registered transform identified by hash $h$ and any input bytes $b$:

$$\texttt{transform}_h(b) = c \quad \forall \text{ platforms}, \forall \text{ time}$$

This determinism enables:

- **Reproducibility:** Any node with the same WASM module and input produces the same output. Results can be independently verified.
- **Caching:** Results can be cached by $(h, b)$ without invalidation concerns from nondeterminism.
- **Distributed verification:** Multiple nodes can independently compute and compare results to detect tampering.

## Claims

1. A computer-implemented method for guaranteeing correctness and safety of user-submitted code executing inside a database, comprising:
   a. receiving a WebAssembly module containing user-submitted transform code;
   b. computing a SHA-256 hash of the module's bytecode to derive a content-addressed identity for the transform;

c. performing registration-time verification comprising type compatibility checking, import resolution, import graph acyclicity verification, and gas limit validation, all before any data flows through the transform;

d. storing the verified module in a registry indexed by its content-addressed hash;

e. at execution time, recomputing the SHA-256 hash of the stored module and comparing to the expected hash, rejecting execution if the hashes do not match;

f. executing the module in a sandboxed WASM runtime with no filesystem, network, clock, or system access, and no access to database records or host memory outside the module's own linear memory.

2. The method of claim 1, wherein the sandboxed WASM runtime enforces the following constraints on the executing module:

   a. no WASI filesystem capabilities are provided, preventing file read or write operations;

   b. no WASI network capabilities are provided, preventing socket or HTTP operations;

   c. no clock or time functions are provided, preventing timing-based side channels;

   d. the module operates exclusively within its own linear memory, which is bounded to a configured maximum size;

   e. the module receives only serialized input bytes for the specific field being transformed and cannot access any other database records, fields, or schemas.

3. The method of claim 1, further comprising gas metering for termination guarantee, wherein:

   a. each module is assigned a maximum fuel budget declared at registration time;

   b. the fuel budget is verified to be within system-configured bounds during registration;

   c. during execution, every WASM instruction consumes fuel from the budget;

   d. when the fuel budget is exhausted, execution immediately aborts with a fuel-exhaustion error;

   e. the fuel budget provides an upper bound on execution time regardless of the module's behavior.

4. The method of claim 1, wherein execution is deterministic such that for any registered transform identified by hash $h$ and any input bytes $b$, the transform produces identical output on every platform and at every time, achieved through:

   a. NaN canonicalization of all floating-point values to a single representation;

   b. absence of system-dependent state access (no clock, filesystem, network, or system calls);

   c. WASM specification guarantees of identical semantics across all conforming runtimes.

5. The method of claim 1, wherein registration-time type verification comprises:

   a. verifying that the transform's declared input type matches the source field's data type;

   b. verifying that the transform's declared output type matches the target field's data type;

   c. rejecting transforms with type mismatches before any data is processed through the transform.

6. The method of claim 1, further comprising reversibility verification, wherein:

a. a transform optionally exports an inverse function in addition to the forward transform function;
b. at registration time, the system generates type-appropriate synthetic test inputs;
c. for each test input, the system runs the forward transform followed by the inverse transform;
d. the system verifies that the round-trip reconstructed input matches the original input, with floating-point tolerance of $1 \times 10^{-10}$;
e. transforms that fail round-trip testing are rejected as non-reversible;
f. verified-reversible transforms enable write-back through the inverse transform to mutate source data.

7. The method of claim 1, wherein data passing between the database host and the WASM module occurs through a defined protocol comprising:

a. the host serializes a typed field value to bytes using a deterministic binary format;
b. the host calls the module's exported `alloc(size)` function to allocate space within the WASM linear memory;
c. the host copies the serialized bytes into the WASM linear memory at the allocated pointer;
d. the host calls the module's exported `transform(ptr, len)` function;
e. the module deserializes the input, computes the transform, serializes the output within its linear memory;
f. the host reads the output bytes from the WASM linear memory and deserializes them to a typed field value;
g. at no point does the module access host memory outside its own WASM linear memory.

8. The method of claim 1, further comprising composable transform imports with directed acyclic graph verification, wherein:

a. transforms declare imports of other transforms via WASM import tables;
b. at registration time, all imported transforms are resolved from the registry;
c. the full import graph, including transitive imports, is verified to be a directed acyclic graph;
d. circular dependencies are rejected at registration time;
e. the composed transform's content-addressed hash includes the hashes of all its imports, making the full computation chain cryptographically verifiable.

9. A system for guaranteeing correctness and safety of user-submitted code executing inside a database, comprising:

a. a database engine configured to store and query data;
b. a content-addressed transform registry configured to store WebAssembly modules indexed by the SHA-256 hash of their bytecode, with associated verified metadata including input/output types, gas limits, reversibility status, and import hashes;

c. a registration-time verification pipeline configured to verify type compatibility, resolve imports, check import graph acyclicity, validate gas limits, and optionally verify reversibility via round-trip testing, all before any data flows through a transform;

d. a sandboxed WASM runtime configured to execute transforms with no filesystem, network, clock, or system access, with gas metering for termination guarantee, NaN canonicalization for deterministic execution, and per-module memory limits;

e. a host-guest data passing subsystem configured to serialize typed field values, allocate space within the WASM module's linear memory, copy data in, invoke the transform, and read results back, without the module accessing host memory;

f. a content verification subsystem configured to recompute the SHA-256 hash of a module's bytecode before every execution and reject execution if the hash does not match the expected identity.

## Abstract

A system and method for guaranteeing correctness and safety of user-submitted code executing inside a database through sandboxed WebAssembly modules with content-addressed verification. Each transform is identified by the SHA-256 hash of its WebAssembly bytecode, providing immutability, tamper detection, deduplication, and public auditability. Transforms execute in a sandboxed runtime with no filesystem, network, clock, or system access, and no access to database records or host memory outside the module's own linear memory. Gas metering assigns each module a fuel budget that limits instruction execution, guaranteeing termination without infinite loops. NaN canonicalization and the absence of system access produce fully deterministic execution — identical input always yields identical output on every platform. A registration-time verification pipeline checks type compatibility, import resolution, import graph acyclicity, gas limits, and optional reversibility via round-trip testing before any data flows through a transform. Data passes between the database host and the WASM guest through a defined protocol using serialization, linear memory allocation, and deserialization. Transforms can import other transforms via WASM import tables, with the import graph verified as a directed acyclic graph and the composition hash including all import hashes for full chain verifiability.

*Prepared for provisional patent filing by Edge Vector Foundation.*
*Filing date: [TO BE DETERMINED]*
*This document constitutes a provisional patent application under 35 U.S.C. 111(b).*